
"ball-in": Tutorial de primeros pasos para tomar contacto con los 8-bits, by GuerraTron26
Esto es un tutorial base para "SMS" (Master System) desde "C" (o "asm") con "c2sms", no olvidar recompilar con "make" o con el "compile.bat" creado.

Author: Juan José Guerra Haba - dinertron@gmail.com - Jun, 2026

Web: <https://guerratron.github.io/>

Categoría: Master System, 8 bits console

License: Free BSD. & Open GPL v.3. Keep credit, please.

Versión: 1.0.0

File: compile.bat -> src/ballIn.c Main Class: N/A :: function main(void)

BALL-IN: Tutorial "A PELO" v1.0

C2SMS nos permite crear un proyecto en **c** que compile para la **Master System**. Nos proporciona una forma más cercana e intuitiva de programar evitando lidiar con direcciones y registros del **z80**.

Vamos a utilizar este tutorial para que nos ayude a entender su funcionamiento básico. Para ello disponemos de unos archivos ya preparados para pegarlos en los directorios correspondientes y reemplazar los existentes.

Para este tutorial vamos a hacer las cosas **directas** llamando a funciones de **basic.h** sin utilizar algunos ayudantes (`simpleSound.h`, `spritesManager.h`, ..) que nos proporciona **c2sms** y que, por otro lado, sería la forma correcta de trabajar.

Pero en tutoriales más avanzados SI que los utilizaremos y nos ayudarán aún más sobre todo con la tarea de los archivos **media** (gráficos, sonidos, música, ..). [ver la documentación de la miniAPI de c2sms]

Para probar el tutorial sólo tenemos que ir paso a paso activando una serie de `#defines` incluidos en **const.h**, compilando y probando en **Emulicious**.

Se han preparado unos **define** para habilitar (con 1) ó deshabilitar (con 0) cada nivel de tutorial donde nos encontremos:

```
/// src/const.h
// -----
// HABILITA O DESHABILITA EL NIVEL DE TUTORIAL (EMPEZAMOS EN TUTO1 CON TODO A 0)
#define _TUTO2_0 0 // CARGA EL FONDO
#define _TUTO2_1 0 // HABILITA EL DISPLAY
#define _TUTO2_2 0 // CARGA EL SPRITE
#define _TUTO3_0 0 // MUEVE COORDENADAS DEL SPRITE
#define _TUTO3_1 0 // ACTUALIZA EL DIBUJADO DEL SPRITE
#define _TUTO4_0 0 // SONIDO BEEP GRAVE
// -----
```

REQUISITOS

- **SDCC** instalado en el sistema y la orden **Make** para la línea de comandos.

- Para probar los juegos algún emulador como **Emulicious**.
 - Herramientas para trabajar con archivos **C** o **ASM**.
 - Algunas herramientas extras vendrían bien para convertir gráficos o música a arrays "c". En la carpeta **/tools** vienen algunas sugerencias.
-

TUTO 1: INICIO

Partimos de la base que hemos ejecutado **c2sms.bat** y hemos generado un proyecto en una carpeta determinada, pongamos para este tutorial que al proyecto lo hemos llamado **balln**.

Abrimos la carpeta creada **projects/balln** con **VSC** (Visual Script Code). [puede utilizarse el **ide preferido** de cada cual, pero para este tutorial lo haré con éste]. Lo primero recordar actualizar con las rutas correctas en "c_cpp_properties.json" -> "includePath" y "compilerPath" para permitir un trabajo correcto de **intellisense**.

De momento nos olvidamos de los archivos base que encontramos en la carpeta **/inc**, son todos archivos de cabecera que implementan las funciones a bajo nivel o básicas que necesitaremos para todos los proyectos. También se definen una serie de globales que se utilizarán casi siempre (vblank_ocurrido, _MASTER, _DELTA, _PAUSE, ...) y algunas estructuras útiles. Estos archivos de momento no haría falta tocarlos, en tutoriales más avanzados implementaremos alguna función extra de utilidad.

En la carpeta **src** se encuentran los archivos principales que se necesitarán modificar para el código, hay otros en la carpeta **assets** para los gráficos con su estructura particular, pero ya los veremos más adelante.

Antes de nada descomprimos **files.zip** y copiamos los archivos a las siguientes rutas del proyecto:

- FONDO: **bg_template_def.h** -> assets/bg/
- SPRITE: **ball_def.h** -> assets/sprites/
- MAIN: **balln.c** -> src/
- CONSTANTES: **const.h** -> src/
- LOGO: **logo.png** -> ./, assets/

De momento nos encontramos con el archivo principal en **c** con el nombre de nuestro proyecto. ¿Qué nos encontramos?, pues una serie de **includes** que cargan funciones de más bajo nivel para tenerlas a disposición en nuestro código, algunas definiciones si queremos de variables locales, ... Pero lo más importante son las funciones mínimas que hay que implementar para conservar el *ciclo de vida* de nuestra app:

- **init()** : "configuración de variables y banderas de inicio". Nosotros vamos a precargar aquí nuestros gráficos comunes.
- **draw(..)** : dibujamos, osea, mandamos comandos a direcciones del VDP. [50-60 veces por segundo]
- **update(..)** : realizamos cálculos, movimientos, animaciones, ... [50-60 veces por segundo]
- **update_fast()** : pueden realizarse otros cálculos más rápidamente, o detección de botones. Va al ritmo de la cpu (Z80)

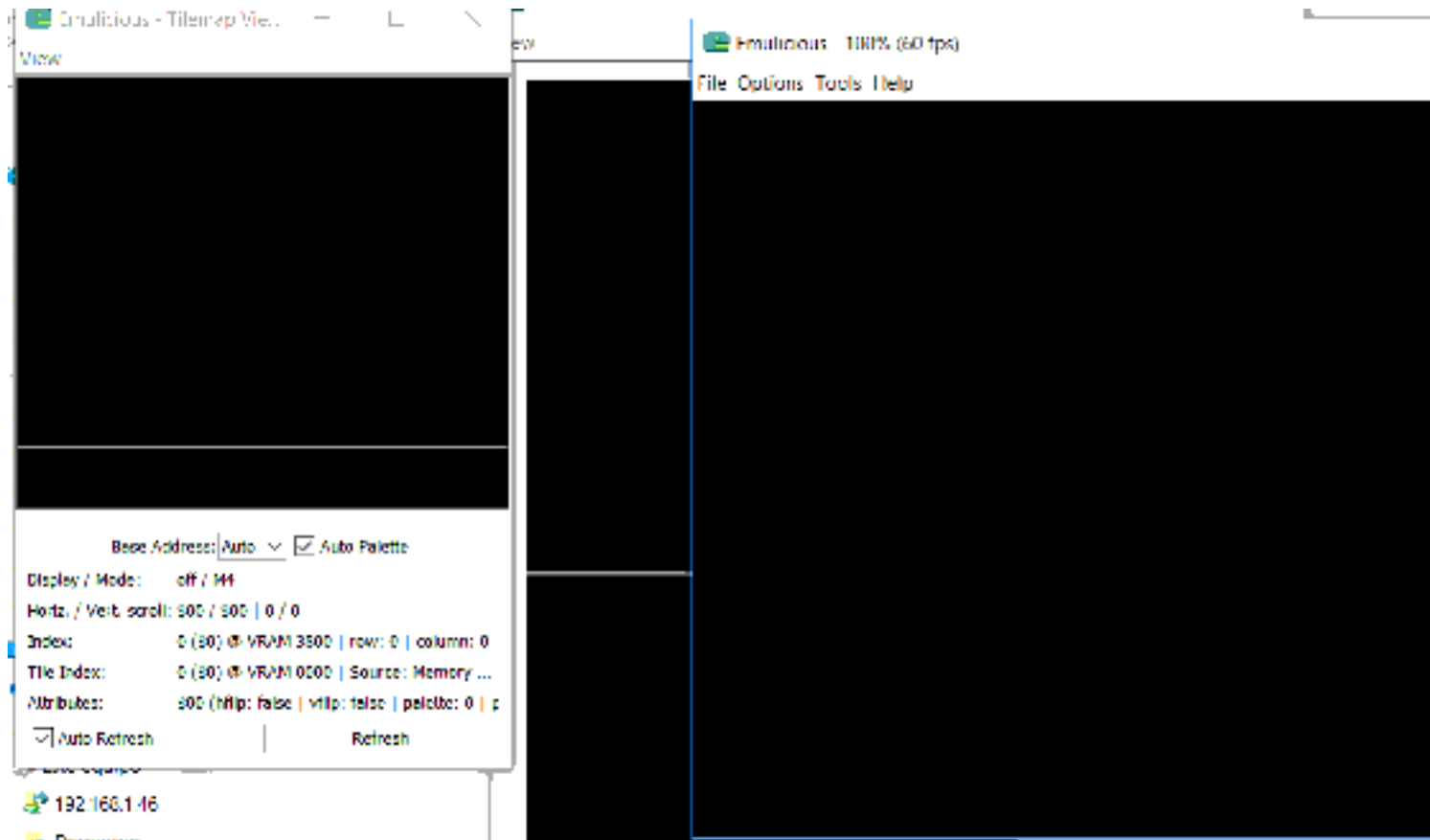
Para entendernos, el mencionado ciclo de vida es un bucle sin fin que irá llamando a **update_fast (muchísimas veces), 1 draw y 1 update** y volverá a comenzar **update_fast (muchísimas veces), 1 draw y 1 update**, ... indefinidamente!.

No hay que preocuparse de interrupciones como V-Blank porque eso ya lo hace por nosotros los archivos cargados en los include.

Así que esto se parece a cualquier ciclo de vida de cualquier juego en cualquier language, de eso se trataba!.

Pero una cosa: Esto hay que inicializarlo de alguna manera, y esto se hace con la función **main()** que se encarga de inicializar y arrancar el bucle. De momento no habría que tocarla, tal cual está nos sirve perfectamente.

Si ejecutamos **make** (o **compile.bat**) ya tendremos nuestro juego de 32Kb preparado en la carpeta de salida **/bin**, que incluso podremos ejecutarlo en un emulador (como **Emulicious**) y tendremos un pedazo de juego que no hace nada, pantalla negra sin música, ni movimientos, ni **nada visual**. Pero en realidad sí realiza todo el ciclo de vida completo, lo que pasa es que no hemos cargado aún nada.



El juego, aunque vacío de gráficos, pesa 32Kb porque así se ha definido en el **start-up** de arranque "crt0sms.s".

TUTO 2: GRÁFICOS

!! REPITO: Se han preparado unos **define** para habilitar (con 1) ó deshabilitar (con 0) cada nivel de tutorial donde nos encontremos:

```
/// src/const.h
// -----
// HABILITA O DESHABILITA EL NIVEL DE TUTORIAL (EMPEZAMOS EN TUTO1 CON TODO A 0)
#define _TUTO2_0 1 // CARGA EL FONDO
#define _TUTO2_1 0 // HABILITA EL DISPLAY
#define _TUTO2_2 0 // CARGA EL SPRITE
#define _TUTO3_0 0 // MUEVE COORDENADAS DEL SPRITE
#define _TUTO3_1 0 // ACTUALIZA EL DIBUJADO DEL SPRITE
// -----
```

Tomando como referencia el archivo `bg_template_def.h` en la carpeta `assets/bg` tenemos definidos tres

arrays: una **paleta** (`_bg_template_pal`), un **tileset** (`_bg_template_til`) y un **tilemap** (`_bg_template_tilemap`).

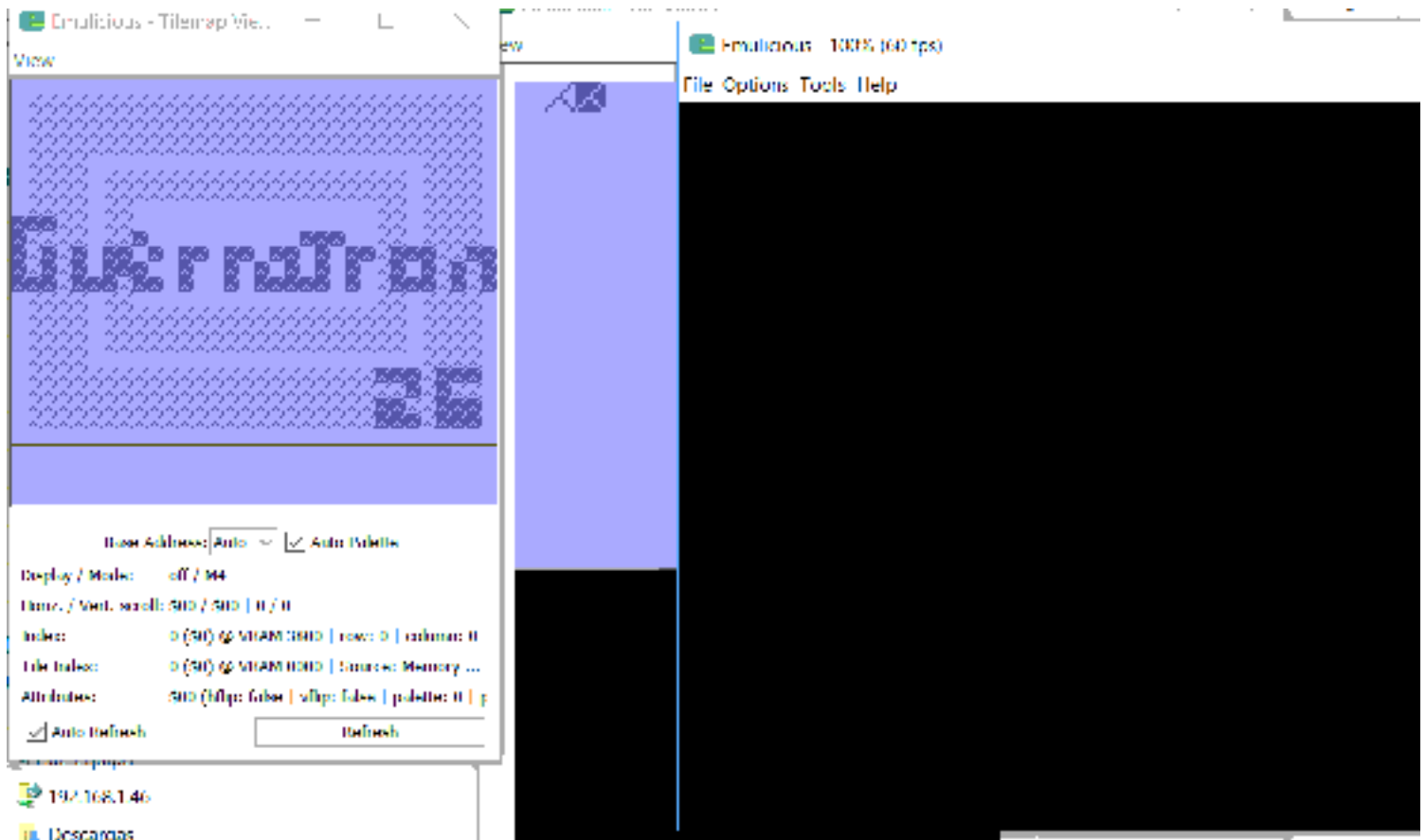
Vamos a cargarlos dentro de la función **init()** "a pelo" con tres funciones que vienen definidas en "inc/basic.h":

FONDOS

```
/// src/const.h
...
#define _TUTO2_0 1 // CARGA EL FONDO
...
/// src/ballIn.c
...
void init(void){
    // FONDO
    write_palette(PALETTE_OFFSET_TILES, _bg_template_pal); // carga la paleta
    write_vram(_bg_template_til, _bg_template_size, 0); // graba los tiles
    write_vram_2(_bg_template_tilemap, _bg_template_tilemap_size, 0x3800); // dibuja en pantalla
}
...
```

Volvemos a recompilar con **make** y ...

- Resultado: La misma pantalla que antes de cargar nada!, !Pero Bueno, ¿Qué ha pasado?; .. Bueno a decir verdad sí que han pasado cosas, si nos vamos en el menú de Emulicious a paletas, tilesets y tilemap vemos que realmente SI están cargados, lo que pasa es que no se muestran en la pantalla.



Para poder mostrar los gráficos en la pantalla tenemos que **habilitar el display**, así que vamos a modificar el código anterior de la función `init()`

```
/// src/const.h
...
#define _TUTO2_1 1 // HABILITA EL DISPLAY
```

```

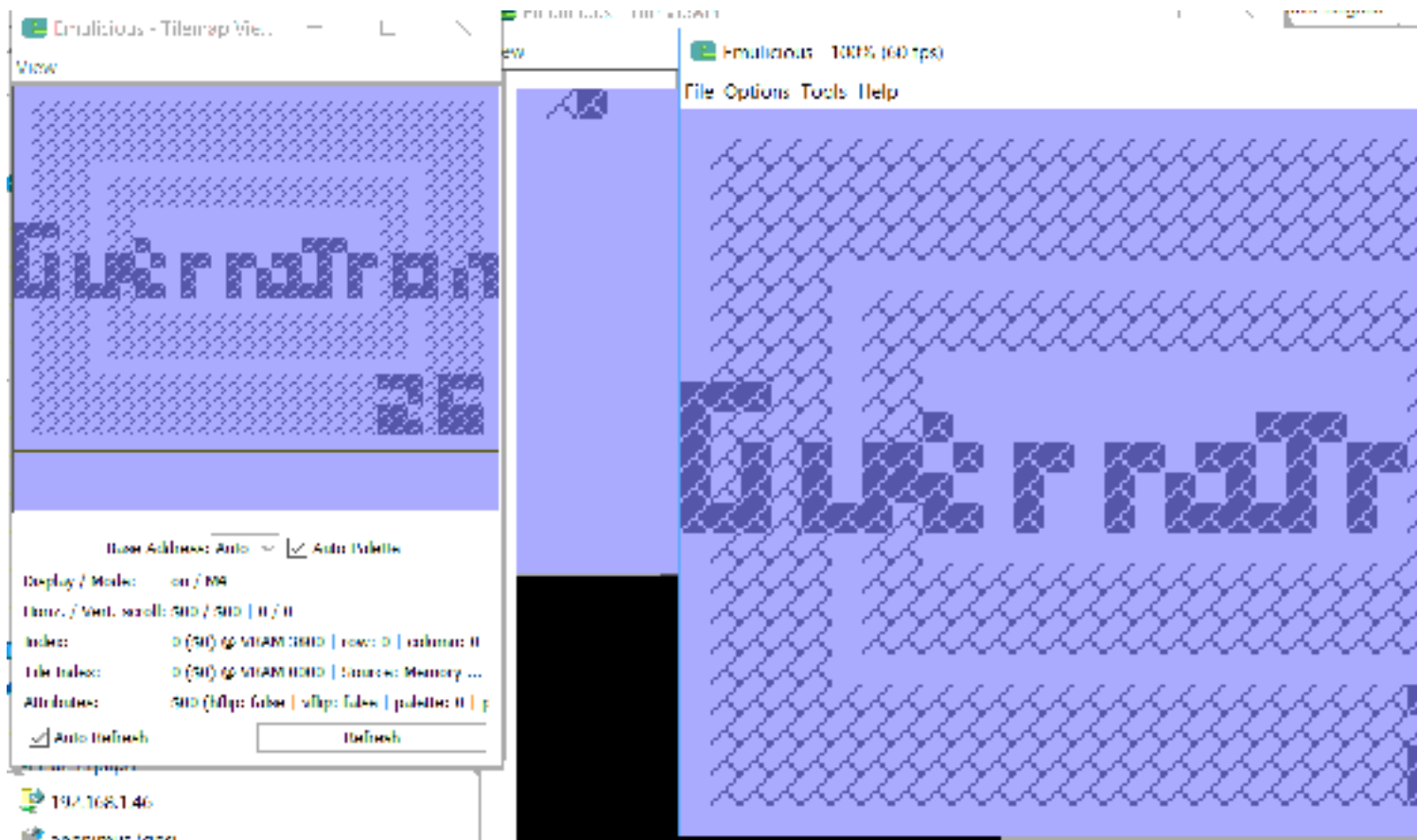
...
/// src/ballIn.c
...
void init(void){
    __asm__("di"); /// Des-Habilitar interrupciones
    // enable display (0x8120)
    VDP_ADDRESS = 0b00100000; // 0x20; // 32
    VDP_ADDRESS = 0x81;
    // FONDO
    write_palette(PALETTE_OFFSET_TILES, _bg_template_pal); // carga la palet.
    write_vram(_bg_template_til, _bg_template_size, 0); // graba los tile.
    write_vram_2(_bg_template_tilemap, _bg_template_tilemap_size, 0x3800); // dibuja en pant.

    // enable display (0x8160)
    VDP_ADDRESS = 0b01100000; // 0x60; // 96
    VDP_ADDRESS = 0x81;
    __asm__("ei"); /// Habilitar interrupciones
}
...

```

AHORA SÍ: Recompilamos con **make** y ya podemos ver un precioso fondo de pantalla *índigo* creado con dos únicos tiles cargados en memoria.

las instrucciones [MACROS] `__asm__` son una forma de pasarle al compilador **SDCC** código ensamblador directamente desde **C**



SPRITES

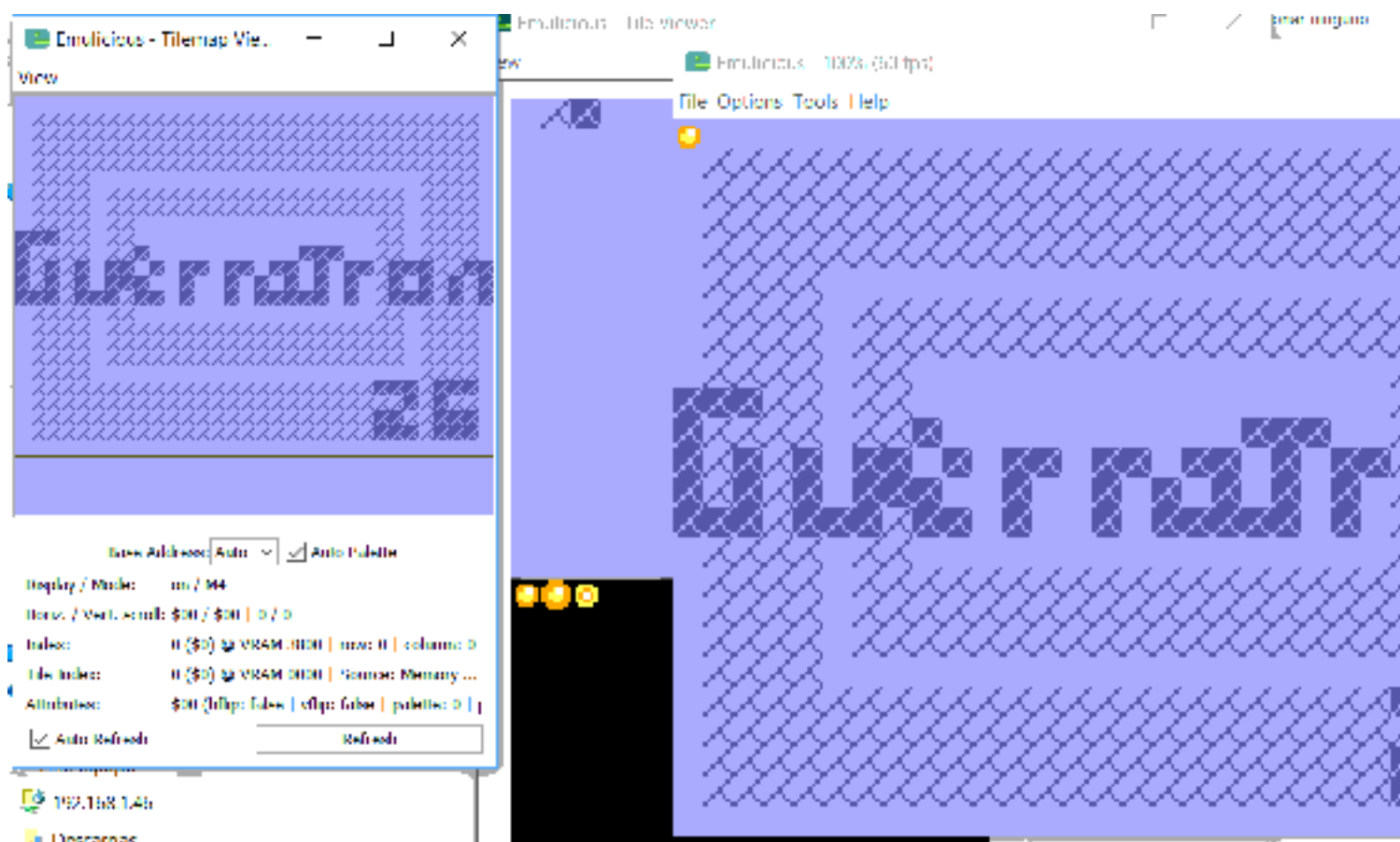
Vamos a probar a cargar sprites. Los sprites van a una zona de memoria distinta y tienen su propia paleta gráfica (aunque en realidad también puede hacer uso de ella el fondo), así que se utilizan las mismas funciones pero con direcciones distintas.

Para ello necesitamos el archivo de definiciones del sprite, que es un archivo con una estructura especial (ya

hablaremos más adelante de ellos). Este archivo es `ball_def.h` en la carpeta `assets/sprites/` que define una simple bolita con tres estados (tiles) que nos permitirían más adelante crear animaciones (similar a **spritesheet**).

De momento utilizaremos uno sólo de estos tres estados cargándolos en memoria. Dentro de la misma función `init()` y siguiendo el código del **FONDO** que ya teníamos (respetando la habilitación y deshabilitación del display al inicio y final):

```
/// src/const.h
...
#define _TUTO2_2 1 // CARGA EL SPRITE
...
/// src/ballIn.c
...
void init(void){
    ...
    // FONDO
    ...
    // SPRITES
    write_palette(PALETTE_OFFSET_SPRITES, _pal_base); // carga la paleta de sprites
    write_vram(_ball_til, _ball_size, _VRAM_SPRITE_PATT); // carga los tiles del sprite en ...
    ...
}
...
```



Ahora también hemos dibujado una pequeña bolita en la esquina *superior-izda* (coordenada 0,0), ¡**Toma ya, nuestro primer Sprite!**!

Si nos fijamos en la zona de paletas también hemos cargado una nueva paleta para sprites con `_pal_base`, que será la que utilizaremos para todos los sprites de este tutorial.

Y en una zona de la **VRAM** para sprites también vemos que se han cargado los tres tiles que componen el array `_ball_til`, de los cuales estamos utilizando el primero de ellos.

la VRAM es la zona reservada de memoria de video para gráficos del VDP (Video Display Processor). Los primeros 256Kb se reservan (o suelen utilizarse) para tilesets de fondos, y los siguientes 256Kb para tilesets de sprites (más o menos).

TUTO 3: MOVIMIENTO

Hasta ahora sólo hemos trabajado en estático en la zona de **carga o configuración** con la función `init()`, si queremos algo de dinamismo (como movimiento, control de pulsación de botones, detección de colisiones, ...) tenemos que trabajar dentro del **LOOP**; osea basicamente `draw()` y `update()`.

Vamos a hacer un pequeño código para manejar el movimiento de la bola.

En nuestro archivo principal `ballIn.c` antes de la función `draw()` crearemos una simple función `moveBall()` para actualizar las coordenadas de nuestra bolita con cada llamada, controlando por supuesto que no salga de los límites de la pantalla, como si estuviera rebotando infinitamente, y la llamaremos dentro de la función `update()`.

Para esto crearemos también cuatro variables globales que almacenen las coordenadas en cada actualización y las iniciamos en la función `init()`.

```
/// src/const.h
...
#define _TUTO3_0 1 // MUEVE COORDENADAS DEL SPRITE
...
/// src/ballIn.c
...
// variables globales de movimiento autoexplicativas
uint8_t xBall = 0;
uint8_t yBall = 0;
uint8_t xIncr = 1;
uint8_t yIncr = 1;
...
void init(void){
    ...
    // posición de inicio
    xBall = 0;
    yBall = 0;
    xIncr = 1;
    yIncr = 1;
}
...
/** controla el movimiento de la bolita sin salirse de los límites. Retorna "true" si se sale
bool moveBall(void){
    bool result = false;
    xBall += xIncr;
    if ((xBall >= _SCREEN_WIDTH - 8) || (xBall <= 0)){
        xIncr *= -1;
        result = true;
    }
    yBall += yIncr;
    if ((yBall >= _SCREEN_HEIGHT - 8) || (yBall <= 0)){
        yIncr *= -1;
        result = true;
    }
    return result;
}
...
void draw(uint32_t delta, uint8_t master)...
...
void update(uint32_t delta, uint8_t master){
    moveBall(); // más tarde utilizaremos su valor de retorno
```

```
}  
...
```

NO!!, espera, no ejecutes nada todavía, aún no los hemos representado en la función `draw()`.
Queda lo más difícil, situar las **x** e **y** en la **tabla de sprites**. Un Galimatías, ya te digo yo!.

Ahora vamos a modificar esa tabla que reside en memoria (SAT) a **pelo**, pero esto no es necesario utilizando ayudantes más avanzados que vienen incluidos en **c2sms**.

De momento mejor que te creas lo siguiente sin más:

```
/// src/const.h  
...  
#define _TUTO3_1 1 // ACTUALIZA EL DIBUJADO DEL SPRITE  
...  
/// src/ballIn.c  
...  
void draw(uint32_t delta, uint8_t master){  
    // Solo permite la primera posición 'y' (el primer tile)  
    VDP_ADDRESS = (uint8_t)(_VRAM_SPRITE_INFO_Y & 0x00FF) + 0; // byte bajo  
    VDP_ADDRESS = 0b01000000 | ((uint8_t)((_VRAM_SPRITE_INFO_Y >> 8) & 0x3F)); // byte alto  
    VDP_DATA = yBall; // coordenada Y  
    VDP_DATA = _VRAM_SPRITE_END; // end of sprite list (sprite_y = 208)  
    // X  
    // Posición X e Índice: Base 0x3F80 (\(0x7F80\) para escribir)  
    VDP_ADDRESS = (uint8_t)(_VRAM_SPRITE_INFO_X & 0x00FF) + 0; // byte bajo  
    VDP_ADDRESS = 0b01000000 | ((uint8_t)((_VRAM_SPRITE_INFO_X >> 8) & 0x3F)); // byte alto  
    VDP_DATA = xBall; // coordenada X  
    VDP_DATA = 1; // AHORA MUESTRA EL TILE 1, OSEA EL SEGUNDO  
}  
...
```

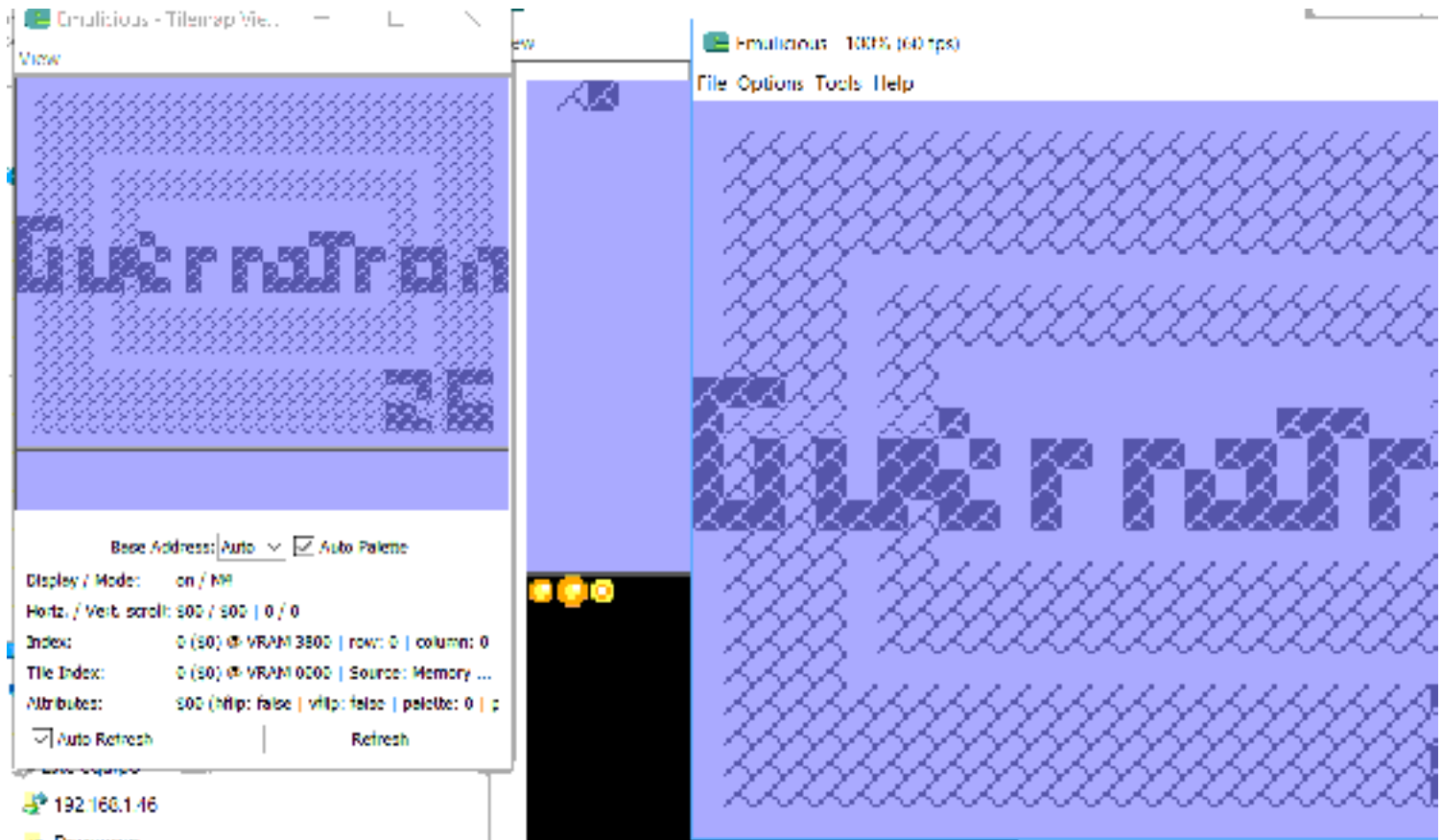
Vamos a ver si explicamos esto por encima:

Los tiles de los sprites se cargan en memoria, como los del fondo, pero en una región diferente; además tienen como dijimos una **SAT** (*Sprite Attribution Table*) que está en otra zona de memoria `0x3F00` y es donde se escriben sus propiedades, como sus coordenadas y el índice del tile a reproducir. Para cada sprite (admite hasta 64 en cada frame) se almacenan 3 bytes NO CONSECUTIVOS.

El lío es que las coordenadas **y** se cargan en una zona y las **x** y el **index** en otra, además hay que marcar la finalización de la última coordenada **y**.

Un poco lioso, la verdad.

Pero bueno, el resultado es un sprite con movimiento. Genial, ¿o No?



OBSERVACIONES:

Fijaos en la parte del **Sprite Viewer** ahora a diferencia de antes se representa un único sprite, esto es porque le hemos situado la finalización ahí con la línea: `VDP_DATA = _VRAM_SPRITE_END; // end of sprite list (sprite_y = 208).`

Además podemos observar un poco más abajo cómo sus coordenadas X/Y se van actualizando rapidísimamente.

Otra cosa es que ya no representa la misma imagen que en el anterior paso ya que le hemos indicado expresamente que muestre el **tile n. 1** en vez del **tile n. 0** con la línea: `VDP_DATA = 1; // AHORA MUESTRA EL TILE 1, OSEA EL SEGUNDO`

Fijaos lo útil que resultan las herramientas de **Emulicious**, no es sólo un simple emulador, tiene viewers, memory maps, depurador, ... ¡ es Magnífico !

TUTO4: SONIDOS

Vamos a dar un aporte más a este minitutorial. Algún efecto sonoro que dé viveza a esa bola en movimiento.

Para sonidos y música en la SMS se utiliza el formato **VGM** (Video Game Music) que habla directamente con el **PSG** (*Programmable Sound Generator*) aunque existen herramientas para obtenerlo desde otros como los **MOD** de la **Amiga** y los **SID** del **Comodore**.

Nosotros no vamos a generar música, para eso podríamos utilizar melodías libres ya creadas, o generarlas con ayuda de algún **tracker**, **synthetizer** o similar como **DefleMask**, **Chiptone**, **OpenMPT**, ... Simplemente generaremos unos simples comandos de sonido, a modo de **beeps** para simular un golpeo de la pelota.

Para esto podemos crear una nueva función justo después de `moveBall()` y llamarla desde el `update()`

```

/// src/const.h
...
#define _TUTO4_0 1 // SONIDO BEEP GRAVE
...
/// src/ballIn.c
...
bool moveBall(void)...
...
/** crea un pequeño sonido "beep" */
void beep(void){
    // 1. Configurar Tono del Canal 0
    PSG = 0x8B; // Byte bajo de tono
    PSG = 0x09; // Byte alto de tono
    // 2. Configurar Volumen del Canal 0 (0 = Máximo, 15 = Mute)
    PSG = 0x90; // 1001 0000 (Canal 0, Volumen Máximo)
    // 3. Esperar un momento (dejar sonar el beep)
    delay(100);
    // 4. Silenciar el Canal 0 (Volumen 15 = Silencio)
    PSG = 0x9F;
    // 3. Esperar un momento (en silencio)
    delay(100);
}
...
void update(uint32_t delta, uint8_t master){
    if(moveBall()){
        beep();
    }
}
...

```

Hala!, ya tenemos una bolita rebotando con sonido. !Podemos vender el juego y forrarnos!

ANOTACIONES

Ya había comentado que todo esto sería una simple **toma de contacto** para ir consolidando conocimientos.

Seguro que la parte de **TUTORIAL 3: MOVIMIENTO** ha resultado compleja, es normal porque estamos tratando con comandos a más bajo nivel; pero todo esto **ya no va a ser necesario** en el siguiente tutorial.

Precisamente para no tener que lidiar nosotros mismos con todos los registros y comandos del **Z80 + VDP + PSG** es por esto por lo que he creado esta herramienta **C2SMS** que lo trata todo de forma **semi-automatizada**. Gracias a que utiliza unos objetos con propiedades particulares que junto a un puñado de funciones de inicialización y archivos de definiciones de gráficos lo construiremos todo mucho más fácil.

Ya había comentado al principio sobre ello, pero a partir de aquí trataremos con los **ayudantes** que nos facilitarán aún más sobre todo con la tarea de los gráficos. [ver la documentación de la miniAPI de c2sms]

Ahora podemos avanzar al **TUTORIAL AVANZADO de C2SMS** y ya verás como se trabaja de una forma más cómoda.

Aunque recomiendo leer o recurrir a la **miniAPI DOCUMENTACIÓN** para entender mejor el sistema sobre el que trabaja **C2SMS** y el puñado de archivos de cabecera sobre el que se apoya para simplificar las cosas.

FIN DEL TUTORIAL "A PELO"

USO RÁPIDO

Después de modificar el código **C** ir a la ventana de comandos y ...

> make

... voilà ! ya tenemos nuestro archivo **.sms** en el directorio "**bin**"; incluso puede probarse el ".sms" generado en un emulador como "*emulicious*".

... el siguiente pasito sería utilizar las funciones y metodología del **TUTORIAL AVANZADO** y mejorar el código "**c**" (o los valientes el "**asm**") y "*re-compile*"

EXTRAS

Mirar en la carpeta "tools", en mi pc tengo enlaces a pequeñas herramientas, pero evidentemente cada cual incluirá aquí lo que prefiera.

Por ejemplo se aporta algún enlace a herramientas más externas como **Img2SMS** o **TilesZipViewer** que nos ayudan con la tarea de convertir imágenes a un formato compatible con la Master System. O **retilesify** que ayuda a desplazar los tilesets con el color de la paleta seleccionada. También **checksumfix** que repara la firma del checksum obligatorio en la cabecera de los **sms**

AGRADECIMIENTOS

Antes de nada mi reconocimiento y Mil-Gracias a todos los *Masters* del código que nos han ayudado con sus útiles herramientas y sus conocimientos, en particular **Maxim, Bock, Avelino Herrera, Calindro, sverx, SteveProXNA, toxa, kusfo ...** y otros mil maestros más. Y por supuesto también a webs como "<https://www.smspower.org/>", "<https://avelinoherrera.com/blog/index.php?m=06&y=25>", "<https://raelcunha.com/2016/07/29/my-retro-journey-sega-master-system/>", "<https://steveproxna.blogspot.com/2017/09/devkitsms-programming-setup.html>", "https://www.elotrolado.net/hilo_tutorial-muy-basico-de-c-para-master-system_2208028" y todos los sitios donde he leído tutoriales o he aprendido algo.

QUE QUEDE CLARO !!!

- Esto **ES** un tutorial a título orientativo (para tomar contacto) de cómo desarrollar un juego o app sencillo para la *Master System I*, sin dependencias externas y con las herramientas que utilizo YO. Simplemente por si puede servirle a alguien de ayuda.
- Esto **NO** es la mejor manera de crear estos juegos, existen herramientas especializadas infinitamente mejores como **devKitSMS, SMSLib, CrossZGB, GBDK-2020, ...** y seguramente muchas más, de gente que sabe **muchiiiiisimo** más que yo sobre este tema. Yo sólo soy un *aprendiz de brujo* que disfruto programando en mis ratos libres.
- Esto **NO** es un tutorial para aprender "buenas técnicas de programación en **C** ni **ASM**", ni cómo utilizar los mejores estándares y paradigmas de programación, ni tampoco programación avanzada.
- Esto **NO** es un tutorial para aprender todas las características técnicas de una máquina como la **SMS**. Tampoco sabría dar detalles sobre el **start-up** de la ROM, ni muchos otros tecnicismos.

Para profundizar en conocimientos técnicos sobre la forma correcta de programación en estas videoconsolas de 8 Bits sería mucho mejor seguir tutoriales de gente como la mencionada al principio.

YO **NO** soy ningún experto, como los mencionados anteriormente (OJALÁ algún día!) De hecho asumo que puedo estar equivocado en las conclusiones que he ido sacando de mis experiencias con la programación y que algunas explicaciones técnicas quizás no se ajusten completamente a la realidad, en tal caso LO SIENTO MUCHO!.

¡ Yo sólo sé QUE NO SÉ NADA !

hecho con amor por GuerraTron26. dinertron@gmail.com